Guía de estudio — Shared Memory + Semaphores (carpetas 10 y 11 del repo)

> Repo: `pal_control_os_study` (GitHub, rama con las etapas nuevas). Cubre `10_shared_memory_semaphores/` (progresión de ejemplos) y `11_controles_shm_semaphore/` (7 simulacros de control). Mismo formato que las guías anteriores: primero sintaxis nueva línea por línea, luego el recorrido por la progresión completa, explicando para qué sirve, cómo se usa y cómo funciona cada pieza.

Índice

1. Idea central antes de entrar al código
2. Sintaxis nueva, línea por línea (usando los archivos 01 y 02)
3. Progresión completa de `10_shared_memory_semaphores`
4. Los 7 controles de `11_controles_shm_semaphore`
5. Caso resuelto completo: Control 07 (integración SHM + SEM + MQ)
6. Checklist de errores típicos
7. Comandos clave
8. Preguntas de sustentación que trae el propio repo

---

1. Idea central antes de entrar al código

`[contexto general]` Hasta ahora (pipes, FIFO, Message Queues) los procesos se comunicaban **pasándose datos** de uno a otro: alguien escribe, alguien lee, el dato "viaja". Shared Memory es distinto: **no hay viaje**. Varios procesos comparten literalmente el mismo bloque de memoria física — cuando uno escribe ahí, los demás ven el cambio de inmediato, sin que nadie tenga que leer ni escribir "a través" de nada.

Eso es rapidísimo, pero trae un problema nuevo que con pipes no existía: si **dos procesos escriben al mismo tiempo** en esa memoria compartida, pueden pisarse el uno al otro (una *race condition* / condición de carrera). Un semáforo (*semaphore*) resuelve exactamente eso: es un contador que el kernel mantiene de forma atómica, y que sirve para que un proceso le diga a los demás "espera, estoy usando este recurso" y luego "ya terminé, puedes pasar".

La frase que resume toda la etapa (está en el propio repo, `COMANDOS_IPC_Y_DEPURACION.txt`):

```
shmget()/shmat()  -> compartir DATOS.
semget()/semop()  -> SINCRONIZAR acceso a esos datos.

Un semáforo no transporta el mensaje.
La memoria compartida no evita por sí sola que dos procesos escriban al mismo tiempo.
```

El patrón que se repite en **todos** los archivos de esta etapa:

```
crear/obtener SHM -> shmat()
crear/obtener SEM -> semget()
inicializar SEM   -> semctl(... SETVAL ...)

P / wait:
    semop(... -1 ...)
    [SECCIÓN CRÍTICA SOBRE LA SHM]
V / signal:
    semop(... +1 ...)

al final:
    shmdt()
    shmctl(... IPC_RMID ...)
    semctl(... IPC_RMID ...)
```

---

2. Sintaxis nueva, línea por línea (usando los archivos 01 y 02)

Estos dos archivos (`01_shared_memory_basica/` y `02_semaphore_basico/`) introducen **toda** la sintaxis que vas a reutilizar en el resto de la etapa. Como ya conoces `pipe`, `fork`, `read`/`write` de las guías anteriores, aquí me enfoco solo en lo que es nuevo.

§2.1 Shared Memory: las 4 funciones

```
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHMSZ 27
#define SHM_KEY 5678

int shmid;
char *shm;

shmid = shmget(SHM_KEY, SHMSZ, IPC_CREAT | 0666);
shm = shmat(shmid, NULL, 0);
/* ... usar shm como memoria normal ... */
shmdt(shm);
shmctl(shmid, IPC_RMID, NULL);
```

Función	Qué hace
`shmget(key, tamaño, flags)`	Crea (con `IPC_CREAT`) u obtiene el **identificador** (`shmid`) de un segmento de memoria compartida de `tamaño` bytes, asociado a `key`. Equivalente a `msgget()` / `pipe()` pero para memoria.
`shmat(shmid, NULL, 0)`	**Adjunta** (*attach*) ese segmento al espacio de direcciones de **este** proceso, y devuelve un puntero (`char *`, o el tipo que necesites) a esa memoria. El `NULL` deja que el sistema elija dónde mapearlo; el `0` son flags (ninguno especial aquí). A partir de esta línea, `shm` se usa exactamente como cualquier arreglo normal.
`shmdt(shm)`	**Separa** (*detach*) el segmento de este proceso — el proceso deja de verlo, pero el segmento sigue existiendo en el kernel (otros procesos todavía lo pueden usar).
`shmctl(shmid, IPC_RMID, NULL)`	**Elimina** el segmento del kernel por completo. `IPC_RMID` es la operación "remove ID". El `NULL` es para una estructura de control que no hace falta aquí. Una vez hecho esto, nadie más puede adjuntarse a ese `shmid`.

`[contexto general]` Fíjate el paralelismo con `pipe`: `shmget` es como "crear el canal", `shmat` / `shmdt` son como "abrir"/"cerrar" ese canal para este proceso en particular, y `shmctl(IPC_RMID)` es como el `unlink()` de un FIFO — borra el recurso del sistema para siempre.

§2.2 `shm == (char *)-1` — por qué se compara así y no con `NULL`

```
shm = shmat(shmid, NULL, 0);
if (shm == (char *)-1) {
    perror("shmat");
    return 1;
}
```

Ya sabes que casi todas las funciones de sistema devuelven `-1` en caso de error (§0.5 de la guía de sintaxis anterior). `shmat()` también, pero como devuelve un **puntero** (`char *`), no puede devolver literalmente el entero `-1` — devuelve el puntero que resulta de "convertir" `-1` a tipo puntero. Por eso la comparación de error se escribe `(char *)-1` en vez de `NULL`: es la forma en que esta función en particular señala "falló".

§2.3 Usar la memoria compartida como un arreglo de `char`

```
char *s;
char c;

s = shm;
for (c = 'a'; c <= 'z'; ++c) {
    *s++ = c;
}
*s = '\0';
```

- `s = shm;`: `s` es un puntero auxiliar que arranca apuntando al mismo lugar que `shm`. Se usa `s` en vez de mover `shm` directamente para conservar la dirección original (la vas a necesitar después para `shmdt`).

- `for (c = 'a'; c <= 'z'; ++c) { *s++ = c; }`: recorre las 26 letras del alfabeto. `*s++ = c` hace dos cosas en una línea: primero escribe `c` en la posición que apunta `s` (`*s = c`), y **después** avanza `s` una posición (`s++`). Es la forma compacta de "escribir y avanzar" que vas a ver todo el tiempo en C.
- `*s = '\0';`: después del bucle, `s` quedó apuntando justo después de la `z`; ahí se escribe el terminador de cadena, igual que en `programa2.c` de las guías anteriores — así el otro proceso puede tratar esto como una cadena normal con `printf("%s", ...)`.

§2.4 El protocolo de confirmación "artesanal" (sin semáforo todavía)

```
/* servidor */
while (*shm != '*') {
    sleep(1);
}

/* cliente */
*shm = '*';
```

`[contexto general]` Esto se llama *polling* (consulta activa): el servidor no tiene forma de "enterarse" automáticamente de que el cliente ya escribió algo, así que simplemente revisa el primer byte una y otra vez, durmiendo 1 segundo entre revisión y revisión, hasta que encuentra el carácter `'*'`. El propio repo lo marca como un mecanismo "artesanal" — funciona, pero gasta CPU innecesariamente y no sirve para proteger datos complejos. Es el ejemplo que justifica por qué hace falta un semáforo de verdad (lo ves en el archivo 02).

§2.5 `union semun` — por qué existe esta unión

```
union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};
```

`[contexto general]` Una `union` (a diferencia de un `struct`) es un tipo donde **todos los campos comparten el mismo espacio de memoria** — en cualquier momento, solo uno de ellos tiene sentido. `semctl()` necesita un cuarto argumento cuyo "significado" cambia según qué operación le pidas: si le pides `SETVAL` (fijar un único valor), espera un `int` (`val`); si le pides `SETALL` (fijar varios valores de un conjunto de semáforos de una vez), espera un arreglo (`array`); si le pides información administrativa, espera un `struct semid_ds *` (`buf`). En vez de tener tres funciones distintas, POSIX definió una sola función que recibe esta "caja" que puede contener cualquiera de las tres cosas. **Nota:** en Linux moderno a veces esta unión ya viene definida en `<sys/sem.h>`; el repo la define manualmente porque en algunos sistemas (POSIX estricto) hace falta declararla tú mismo — no está de más tenerla.

§2.6 Crear e inicializar un semáforo: `semget` + `semctl(SETVAL)`

```
#include <sys/sem.h>

#define SEM_KEY 0x4444

int semid;
union semun arg;

semid = semget(SEM_KEY, 1, 0666 | IPC_CREAT);

arg.val = 1;
semctl(semid, 0, SETVAL, arg);
```

- `semget(SEM_KEY, 1, 0666 | IPC_CREAT)`: crea (u obtiene) un **conjunto** de semáforos. El `1` es cuántos semáforos tiene ese conjunto (aquí, solo uno). Nota el paralelismo exacto con `msgget` / `shmget`.
- `arg.val = 1;`: usa el campo `val` de la unión — aquí sí tiene sentido porque vas a usar `SETVAL` (fijar **un** valor).
- `semctl(semid, 0, SETVAL, arg)`: de los 4 argumentos, `semid` es el conjunto, el `0` es **el índice del semáforo dentro del conjunto** (aquí solo hay uno, el índice 0), `SETVAL` es la operación ("set value"), y `arg` trae el valor nuevo. El resultado: ese semáforo queda en `1`, lo que por convención significa "recurso disponible / nadie lo está usando".

§2.7 `struct sembuf` y `semop()` — las operaciones P y V

```
static struct sembuf operacion_p = {0, -1, 0};
static struct sembuf operacion_v = {0, +1, 0};

static void semP(int semid) {
    if (semop(semid, &operacion_p, 1) == -1) {
        perror("semop P");
        exit(13);
    }
}

static void semV(int semid) {
    if (semop(semid, &operacion_v, 1) == -1) {
        perror("semop V");
        exit(14);
    }
}
```

- `struct sembuf { ... }`: una estructura con 3 campos: `sem_num` (qué semáforo del conjunto, por posición — aquí `0`), `sem_op` (qué operación: un número negativo resta, uno positivo

suma), y `sem_flg` (banderas opcionales, aquí `0` = ninguna especial). `{0, -1, 0}` es literalmente "al semáforo número 0, réstale 1, sin banderas especiales".
- `semop(semid, &operacion_p, 1)`: ejecuta la operación. El `1` final es "cuántas operaciones hay en el arreglo que le estoy pasando" (aquí solo una; podrías pasarle varias de una vez, aunque en este repo no se usa así). **Esta es la línea que de verdad hace el trabajo**: si el semáforo vale `0`, `semop` con `-1` **bloquea** al proceso hasta que otro proceso lo suba con `+1`; si vale `1` o más, lo resta de inmediato y el proceso sigue sin esperar.
- `semP` / `semV`: son simplemente nombres mnemotécnicos (de *Proberen*/*Verhogen*, los términos clásicos de Dijkstra) para "bajar" (`P`, entrar a la sección crítica) y "subir" (`V`, salir de ella). El repo también las llama `down` / `up` en otros archivos — son el mismo patrón con otro nombre.
- `exit(13)` / `exit(14)`: números de salida arbitrarios distintos, solo para poder distinguir en qué operación falló si algo sale mal (no es un patrón obligatorio, es una elección de estilo de este archivo).

§2.8 El patrón completo P → sección crítica → V

```
for (size_t i = 0; i < strlen(texto); ++i) {
    semP(semid);

    putchar(texto[i]);
    fflush(stdout);
    sleep(rand() % 2);
    putchar(texto[i]);
    fflush(stdout);

    semV(semid);
    sleep(rand() % 2);
}
```

- Todo lo que está **entre** `semP` y `semV` es la "sección crítica": mientras un proceso está ahí dentro, ningún otro proceso que use el mismo semáforo puede entrar a la suya.
- El `sleep(rand() % 2)` **dentro** de la sección crítica es a propósito: agranda la ventana de tiempo en la que, si el semáforo no estuviera funcionando, otro proceso podría "colarse" y mezclar sus caracteres con los tuyos. Es una forma de **demostrar visualmente** que el semáforo funciona (si lo quitas, vas a ver los caracteres de dos procesos distintos entremezclados).
- El `sleep(rand() % 2)` que está **después** de `semV` (fuera de la sección crítica) no protege nada — solo hace la ejecución menos predecible para que el ejemplo se vea más realista.

§2.9 `argv[1][0] == 'c'` — distinguir quién crea y quién solo usa

```
semid = (argv[1][0] == 'c') ? crear_semaforo() : obtener_semaforo();
```

- `argv[1][0]`: el primer carácter del primer argumento (`argv[1]` es un `char *`; `argv[1][0]` es el primer `char` de esa cadena). Se usa para que el mismo ejecutable sirva tanto para

"crear" (`c`) como para "obtener" (`g`, de *get*) el semáforo, sin tener que compilar dos binarios distintos.
- `cond ? valorA : valorB`: el operador ternario — una forma compacta de escribir un `if/else` que **devuelve un valor**. Equivale a: `c if (argv[1][0] == 'c') { semid = crear_semaforo(); } else { semid = obtener_semaforo(); }`
- La diferencia entre `crear_semaforo()` y `obtener_semaforo()`: la primera usa `IPC_CREAT` (y además inicializa con `SETVAL`), la segunda llama `semget(SEM_KEY, 1, 0)` **sin** `IPC_CREAT` — asume que el semáforo ya existe (igual que `programa2.c`/el FIFO lector de las guías anteriores: quien no crea, solo "se conecta" a lo que ya existe).

---

3. Progresión completa de `10_shared_memory_semaphores`

Cada nivel agrega **una sola idea nueva** sobre el anterior — exactamente como en `01_basics` → `02_pairs` → ... de la etapa de IPC con pipes.

Nivel 1 — `01_shared_memory_basica/`: SHM sola, sin semáforo

Ya la viste completa en la sección 2. Un servidor escribe el alfabeto y espera; un cliente lo lee y confirma cambiando un byte. **Sin protección real** — funciona porque solo un proceso escribe a la vez, por construcción del propio ejemplo (no porque el código lo garantice).

Nivel 2 — `02_semaphore_basico/`: semáforo solo, sin SHM

Ya la viste completa en la sección 2. Dos instancias del mismo programa compiten por una sección crítica (aquí, solo `printf` — no hay memoria compartida todavía). Demuestra el mecanismo de sincronización de forma aislada, antes de combinarlo con datos reales.

Nivel 3 — `03_shm_sem_dos_procesos/`: la combinación real

Qué agrega: por primera vez, el semáforo protege **datos** en la SHM, no solo una salida de consola. Y aparece una `struct` compleja en memoria compartida, no solo un `char *`:

```
typedef struct {
    int contador;
    int terminado;
    char mensaje[128];
} DatosCompartidos;
```

```
int shmid = shmget(SHM_KEY, sizeof(DatosCompartidos), IPC_CREAT | 0666);
...
DatosCompartidos *d;
d = shmat(shmid, NULL, 0);
```

- `sizeof(DatosCompartidos)`: en vez de un tamaño fijo a mano (como el `27` del Nivel 1), aquí se le pide a `shmget` exactamente el tamaño real de la estructura — más robusto, porque si agregas un campo a la `struct`, el tamaño se recalcula solo.

- `d = shmat(...)`: el puntero ya no es `char *`, es `DatosCompartidos *` — a partir de aquí se accede con `d->contador`, `d->terminado`, `d->mensaje`, como cualquier puntero a struct.
- El patrón "coordinador crea, cliente modifica" es el mismo que servidor/cliente del Nivel 1, pero ahora **cada acceso a `d`** va envuelto en `down(semid) / ... / up(semid)` — es la primera vez que ves el patrón completo de la sección 1 aplicado de verdad.
- El coordinador espera en un bucle `while(1)` revisando `d->terminado` **bajo semáforo** cada vez (no revisa la memoria directamente sin protección, a diferencia del polling del Nivel 1).

Nivel 4 — `04_shm_sem_tres_procesos/`: `fork()` entra en escena

Qué agrega: ya no son dos programas distintos (servidor/cliente) — es **un solo programa** que se bifurca con `fork()`, y el contador se incrementa **miles de veces** para demostrar la condición de carrera de forma contundente:

```
#define HIJOS 2
#define REPETICIONES 1000
...
for (int i = 0; i < HIJOS; ++i) {
    pid_t pid = fork();
    if (pid == 0) {
        for (int j = 0; j < REPETICIONES; ++j) {
            down(semid);
            ++(*contador);
            up(semid);
        }
        ...
        _exit(0);
    }
}
for (int i = 0; i < HIJOS; ++i) wait(NULL);
printf("...contador final=%d esperado=%d\n", *contador, HIJOS * REPETICIONES);
```

- Nota el **orden**: la SHM y el SEM se crean **antes** del `fork()`, igual que el pipe se creaba antes de `fork()` en las guías de IPC — así ambos hijos heredan automáticamente acceso al mismo segmento (en este caso heredan el puntero `contador` ya adjuntado, porque `shmat()` se hizo también antes del `fork`).
- `REPETICIONES = 1000` por hijo: no es un capricho — con pocas repeticiones, una condición de carrera real **puede no notarse nunca** (los procesos pueden tener suerte y no pisarse). Con miles de repeticiones, si quitaras el `down` / `up`, verías con alta probabilidad que el contador final **no** da `HIJOS * REPETICIONES` exacto.
- `wait(NULL)` llamado `HIJOS` veces: ya lo conoces de las guías de pipes — espera a cada hijo, uno por uno, para no dejar zombies.

Nivel 5 — `05_shm_sem_cinco_procesos/`: mismo patrón, más procesos

Qué agrega: nada conceptualmente nuevo frente al Nivel 4 — es literalmente el mismo patrón con `HIJOS = 4` (padre + 4 hijos = 5 procesos totales) y `REPETICIONES = 5000`. Está aquí porque

varios controles (`control_03`) piden exactamente "cinco procesos totales", y este archivo es la plantilla directa para eso.

Nivel 6 — `06_shm_sem_turnos_cinco_procesos/`: un conjunto de semáforos, no uno solo

Qué agrega: hasta ahora, `semget(KEY, 1, ...)` siempre creaba un conjunto de **un solo** semáforo. Aquí se crea un conjunto de **5**:

```
#define N 5
int semid = semget(SEM_KEY, N, IPC_CREAT | 0666);

unsigned short valores[N] = {1, 0, 0, 0, 0};
arg.array = valores;
semctl(semid, 0, SETALL, arg);
```

- `semget(SEM_KEY, N, ...)`: el segundo argumento ahora es `5`, no `1` — el conjunto tiene 5 semáforos independientes, numerados `0` a `4`.
- `union semun arg; arg.array = valores;`: aquí sí se usa el campo `array` de la unión (§2.5) — porque la operación `SETALL` fija **todos** los valores del conjunto de una sola llamada, a diferencia de `SETVAL` que fija uno solo.
- `semctl(semid, 0, SETALL, arg)`: nota que el `0` aquí **no** significa "el semáforo número 0" como en `SETVAL` — con `SETALL` ese segundo argumento se ignora, porque la operación afecta a todo el conjunto a la vez.
- La idea de "turnos": `S0=1` (disponible), `S1..S4=0` (bloqueados). Cada proceso hace `P` sobre **su propio** semáforo (espera su turno) y `V` sobre el **siguiente** (le pasa el turno al que sigue):

```
static void op(int semid, unsigned short num, short delta) {
    struct sembuf sb = {num, delta, 0};
    semop(semid, &sb, 1);
}

static void ejecutar_turno(int indice, int semid, Estado *e) {
    op(semid, (unsigned short)indice, -1);       /* P sobre MI semáforo */
    e->orden[e->posicion++] = (char)('A' + indice);
    ...
    if (indice + 1 < N) {
        op(semid, (unsigned short)(indice + 1), +1);  /* V sobre el SIGUIENTE */
    }
}
```

- Fíjate que `op()` ahora recibe `num` como parámetro (qué semáforo del conjunto tocar) — a diferencia de `down`/`up` de niveles anteriores, que siempre usaban el semáforo `0` fijo. Esta es la generalización necesaria para trabajar con un **conjunto**, no con un semáforo suelto.
- El resultado garantizado es `ABCDE` en ese orden exacto, **sin usar `sleep()` para forzar el orden** — el orden lo garantiza la cadena de semáforos, no el tiempo. Esto es justo lo que

pide `control_04` más abajo, y es una distinción importante: "*No se permite resolver el orden mediante sleep()*".

Nivel 7 — `07_shm_sem_signal_inicio/`: señal + 2 semáforos con roles distintos

Qué agrega: combina lo que ya sabes de señales (de la etapa anterior del repo) con un conjunto de **2** semáforos que cumplen **roles diferentes**:

```
#define SEM_INICIO 0
#define SEM_MUTEX  1
...
unsigned short valores[2] = {0, 1};
/* SEM_INICIO=0: todos los hijos arrancan BLOQUEADOS.
   SEM_MUTEX=1:  protege el contador, arranca DISPONIBLE. */
```

- Cada hijo hace, en orden, `P(SEM_INICIO)` (espera la señal de arranque) y **luego** `P(SEM_MUTEX) ... V(SEM_MUTEX)` (protege su incremento al contador). Son dos semáforos con propósitos distintos dentro del mismo conjunto — uno es una "compuerta" (*gate*), el otro es el "candado" (*mutex*) de siempre.
- El padre registra `SIGUSR1` con el patrón de bandera que ya conoces (`volatile sig_atomic_t iniciar`), y cuando la señal llega, hace **4** **`V(SEM_INICIO)`** seguidas — una por cada hijo bloqueado:

```
while (!iniciar) {
    pause();
}
for (int i = 0; i < HIJOS; ++i) {
    sem_op(semid, SEM_INICIO, +1);
}
```

- **Punto conceptual clave** (lo pide `control_05` explícitamente): la señal despierta **al padre** (es el mecanismo `pause()`/handler que ya conoces); el semáforo despierta **a los hijos** (es un mecanismo completamente distinto — los hijos nunca reciben la señal, solo están bloqueados en `semop`). Son dos "despertadores" independientes encadenados: señal → padre → semáforo → hijos.

Nivel 8 — `08_shm_sem_fork_execv/`: recuperar SHM/SEM después de `execv`

Qué agrega: hasta ahora, el hijo heredaba el puntero `contador` ya adjuntado (`shmat()` hecho antes del `fork()`). Pero si el hijo hace `execv()`, **pierde todo su código y sus variables** — el puntero heredado deja de existir. La solución: el nuevo programa vuelve a pedir el mismo recurso **por su** **`KEY`**, no por el puntero heredado:

```
/* coordinador_fork_execv.c */
#define SHM_KEY 0x5206
#define SEM_KEY 0x6206
...
for (int i = 0; i < HIJOS; ++i) {
    pid_t pid = fork();
    if (pid == 0) {
        char indice[16];
        snprintf(indice, sizeof(indice), "%d", i);
        char *args[] = {"./worker_shm_sem", indice, NULL};
        execv(args[0], args);
        ...
    }
}
```

```
/* worker_shm_sem.c — un programa COMPLETAMENTE DISTINTO */
#define SHM_KEY 0x5206   /* la MISMA key, hardcodeada de nuevo */
#define SEM_KEY 0x6206
...
int shmid = shmget(SHM_KEY, sizeof(int), 0666);   /* sin IPC_CREAT: solo obtiene */
int semid = semget(SEM_KEY, 1, 0);
int *contador = shmat(shmid, NULL, 0);             /* shmat() de NUEVO, en este proceso */
```

- Esta es la idea central del nivel: **las keys (`SHM_KEY`, `SEM_KEY`) son el "nombre" del recurso a nivel de todo el sistema**, no algo privado de un proceso. Cualquier proceso que conozca la key correcta puede volver a localizar el mismo segmento de memoria o el mismo semáforo con `shmget` / `semget`, **sin necesidad de heredar nada por `fork()`**. Es un paralelismo directo con el "descriptor por `argv`" que ya viste con pipes (Forma B) — solo que aquí, en vez de pasar un número de descriptor, lo que se comparte es la **constante `KEY`**, escrita igual en ambos archivos fuente.
- Esto es justo lo que dice el enunciado de `control_06`: "*demostrar que un programa ejecutado mediante execv puede volver a localizar IPC System V usando sus keys, sin depender de heredar un puntero de shmat()*".

---

4. Los 7 controles de `11_controles_shm_semaphore`

Cada control reutiliza, casi literalmente, uno de los niveles de la sección 3. Esta tabla es el mapa rápido:

Control	Qué pide	Nivel base que debes repasar
`control_01`	SHM servidor/cliente, con `ipcs -m` y limpieza	Nivel 1 (`01_shared_memory_basica`) — mismo código, literalmente
`control_02`	Semáforo como exclusión mutua, 3 terminales simultáneas	Nivel 2 (`02_semaphore_basico`)
`control_03`	SHM + SEM, exactamente 5 procesos totales, 5000 incrementos c/u	Nivel 5 (`05_shm_sem_cinco_procesos`)
`control_04`	5 procesos en orden estricto `ABCDE`, prohibido usar `sleep()`	Nivel 6 (`06_shm_sem_turnos_cinco_procesos`) — set de 5 semáforos
`control_05`	Signal despierta al padre, semáforo despierta a los hijos	Nivel 7 (`07_shm_sem_signal_inicio`) — 2 semáforos con roles distintos
`control_06`	`fork()` + `execv()`; el worker relocaliza SHM/SEM por su key	Nivel 8 (`08_shm_sem_fork_execv`)
`control_07`	Integra los 3 IPC: SHM (estado) + SEM (exclusión) + MQ (transporte de resultados)	Combina Nivel 5 con lo que ya sabes de Message Queues (guías anteriores) — ver desglose completo en la sección 5

Cómo usarlos: por cada control, primero lee solo `ENUNCIADO.txt` e intenta resolverlo con lo que ya sabes del nivel correspondiente; recién después compara con `solucion/`. Las preguntas de sustentación que trae cada control (ver sección 8) son tan importantes como el código — varias piden explicar *por qué*, no solo mostrar que compila.

---

5. Caso resuelto completo: Control 07 (integración SHM + SEM + MQ)

Es el control más completo de la etapa — junta todo lo nuevo (SHM, semáforos) con lo que ya conocías de Message Queues. Vale la pena verlo línea por línea porque resume toda la guía.

Enunciado (resumen): A es coordinador; crea SHM (contador=0), un semáforo binario, y una Message Queue. Crea 4 hijos. Cada hijo incrementa el contador bajo semáforo, y **además** manda a A un mensaje por la cola con su PID y el valor que observó. A recibe los 4 mensajes, espera a los hijos, y limpia los 3 recursos.

```
typedef struct { long mtype; pid_t pid; int valor; } Mensaje;
```

- Ya conoces esta forma de `struct msg` de las guías de Message Queue: el primer campo siempre `long` (aquí `mtype`, el nombre que usa la API real de System V — es el mismo campo que antes llamábamos `type`). Los campos siguientes (`pid`, `valor`) son el "cuerpo" del mensaje — aquí, en vez de solo texto, se manda una estructura con dos números.

```
int shmid = shmget(SHM_KEY, sizeof(int), IPC_CREAT | 0666);
int semid = semget(SEM_KEY, 1, IPC_CREAT | 0666);
int msqid = msgget(MQ_KEY, IPC_CREAT | 0666);
```

- Los 3 recursos IPC se crean con el mismo patrón, cada uno con su propia `KEY` (`SHM_KEY`, `SEM_KEY`, `MQ_KEY`) — son completamente independientes entre sí a nivel de sistema, aunque este programa los use juntos.

```
for (int i = 0; i < HIJOS; ++i) {
    pid_t pid = fork();
    if (pid == 0) {
        Mensaje m = {.mtype = 1, .pid = getpid(), .valor = 0};

        sem_op(semid, -1);
        ++(*contador);
        m.valor = *contador;
        sem_op(semid, +1);

        if (msgsnd(msqid, &m, sizeof(m) - sizeof(long), 0) == -1) {
            perror("msgsnd");
        }
        fflush(stdout);
        shmdt(contador);
        _exit(0);
    }
}
```

- `Mensaje m = {.mtype = 1, .pid = getpid(), .valor = 0};`: inicializa la estructura completa de una vez (ya viste esta sintaxis de inicialización con nombres de campo en la guía de sintaxis anterior, §0.11).
- El orden importa: primero `sem_op(semid, -1)` (entra a la sección crítica), incrementa el contador **compartido**, y **guarda ese valor exacto en su copia local** `m.valor` **antes** de salir con `sem_op(semid, +1)`. Esto es clave: si copiara `*contador` **después** de salir de la sección crítica, otro hijo podría haberlo incrementado de nuevo mientras tanto, y el mensaje reportaría un valor incorrecto — "qué vio cada hijo" solo es confiable si se lee **dentro** de la misma sección crítica donde se escribió.
- `msgsnd(msqid, &m, sizeof(m) - sizeof(long), 0)`: el tercer argumento (`sizeof(m) - sizeof(long)`) es el tamaño del **cuerpo** del mensaje, sin contar el campo `mtype` (que la propia API de System V maneja aparte) — es la forma estándar de calcular ese tamaño para cualquier struct de mensaje, sea cual sea su contenido.

```
for (int i = 0; i < HIJOS; ++i) {
    Mensaje m;
    if (msgrcv(msqid, &m, sizeof(m) - sizeof(long), 1, 0) == -1) {
        perror("msgrcv");
        return 1;
    }
    printf("[A] mensaje de PID=%d: contador observado=%d\n", (int)m.pid, m.valor);
}
```

- A recibe exactamente 4 mensajes (uno por hijo), filtrando por tipo `1` — el mismo patrón de `msgrcv` que ya conoces. Nota que esto ocurre **después** del `for` que creó los 4 `fork()`, es decir, A no espera a que cada hijo termine antes de pasar al siguiente — simplemente procesa los mensajes en el orden en que van llegando a la cola (que no tiene por qué coincidir con el orden en que se crearon los hijos).

```
for (int i = 0; i < HIJOS; ++i) wait(NULL);
printf("[A] contador final en SHM=%d\n", *contador);

shmdt(contador);
shmctl(shmid, IPC_RMID, NULL);
semctl(semid, 0, IPC_RMID);
msgctl(msqid, IPC_RMID, NULL);
```

- Recién después de recibir los 4 mensajes, A espera a los 4 hijos con `wait(NULL)` — el orden "primero mensajes, después `wait`" no es casual: un hijo manda su mensaje y **luego** hace `_exit(0)`, así que para cuando A ya recibió los 4 mensajes, los 4 hijos están a punto de terminar (o ya terminaron) de todas formas.
- La limpieza final libera los **tres** recursos, cada uno con su función específica: `shmctl(IPC_RMID)` para la memoria, `semctl(IPC_RMID)` para el semáforo (nota que aquí no lleva el `NULL` final como argumento porque `IPC_RMID` sobre un semáforo no necesita el cuarto parámetro de `semctl`), y `msgctl(IPC_RMID, NULL)` para la cola.

La distinción que el propio enunciado pide entender:

```
SHM = estado compartido      (el contador vive ahí)
semaphore = exclusión mutua  (evita que dos hijos lo pisen)
MQ = transporte de eventos   (le avisa a A qué pasó, y cuándo)
```

---

6. Checklist de errores típicos

- [ ] `shmat()` se compara contra `(char *)-1`, **no** contra `NULL` (§2.2).
- [ ] `shmget` / `semget` se llaman **antes** de `fork()` cuando el hijo necesita heredar el puntero ya adjuntado; si el hijo hace `execv()`, en cambio, debe volver a pedir el recurso por su `KEY` (Nivel 8).

- [ ] Toda SHM/SEM que cree un proceso debe limpiarse al final: `shmdt()` + `shmctl(IPC_RMID)`, y `semctl(IPC_RMID)` — si el programa termina mal (ej. `kill -9`), los recursos quedan "huérfanos" en el kernel y hay que borrarlos a mano con `ipcrm`.
- [ ] Cada acceso a una variable en SHM que pueda ser tocada por más de un proceso va envuelto en `P()` / `V()` — incluso una simple lectura (ver el `while(1) { down(); listo = d->terminado; up(); }` del Nivel 3).
- [ ] Con un **conjunto** de semáforos (`semget(key, N, ...)` con `N > 1`), cada `struct sembuf` debe indicar `sem_num` correcto — no asumas que siempre es el semáforo `0`.
- [ ] `SETVAL` inicializa **un** semáforo (necesita el índice); `SETALL` inicializa **todo el conjunto de una vez** (usa `arg.array`, ignora el índice).
- [ ] El valor inicial de un semáforo define su significado: `1` = recurso disponible (mutex clásico); `0` = "compuerta cerrada" (alguien más debe hacer `V` para dejarte pasar) — revisa qué patrón pide el enunciado antes de inicializar.
- [ ] No confundir **quién despierta a quién**: una señal solo puede despertar al proceso que la recibe directamente (vía `pause()`/handler); si ese proceso necesita despertar a otros, debe hacerlo él mismo con `V()` sobre un semáforo — la señal nunca "salta" directo a los hijos.
- [ ] El tamaño que le pasas a `msgsnd` / `msgrcv` cuando el mensaje es una `struct` personalizada es `sizeof(struct) - sizeof(long)` (todo el struct, menos el campo `mtype` / `type`).

---

7. Comandos clave

Propósito	Comando
Ver todos los recursos IPC	`ipcs`
Ver solo memoria compartida	`ipcs -m`
Ver solo semáforos	`ipcs -s`
Ver solo colas de mensajes	`ipcs -q`
Borrar un segmento de memoria por ID	`ipcrm -m SHMID`
Borrar un conjunto de semáforos por ID	`ipcrm -s SEMID`
Compilar (no necesita `-pthread` para SHM/SEM de System V)	`gcc -std=c11 -Wall -Wextra -pedantic archivo.c -o programa`
Enviar señal	`kill -USR1 PID`

Patrón de depuración recomendado por el repo: correr `ipcs -m` y `ipcs -s` antes, durante y después de ejecutar el programa — así puedes confirmar que el recurso se creó, que sigue vivo mientras el programa corre, y que desapareció al terminar (si no desaparece, usa `ipcrm` para limpiarlo a mano antes de la siguiente prueba).

---

8. Preguntas de sustentación que trae el propio repo

Estas son las preguntas que los propios `ENUNCIADO.txt` incluyen — ensaya la respuesta antes de mirarla, están pensadas para que las expliques en el control, no solo para que el código compile.

▼ ¿Qué ocurriría si se retiran P y V del contador compartido? (control_03)
Dos o más procesos podrían leer el mismo valor del contador, incrementarlo cada uno en su copia local de CPU, y escribir de vuelta — perdiendo uno de los dos incrementos. El resultado final sería menor al esperado (`HIJOS * REPETICIONES`), y además no sería reproducible: variaría entre ejecuciones.

▼ ¿Por qué el contador debe estar en SHM y no en una variable normal? (control_03)
Después de `fork()`, cada proceso tiene su propia copia privada de la memoria (incluidas las variables globales/locales normales) — modificar una variable normal en el hijo no afecta en absoluto la copia del padre ni la de otros hijos. Solo la memoria explícitamente compartida con `shmget` / `shmat` es vista por todos los procesos a la vez.

▼ ¿Cuántas veces se ejecuta fork() en el padre, y cuál es el contador final esperado? (control_03)
Se ejecuta tantas veces como hijos se necesiten (4, para llegar a 5 procesos totales incluyendo al padre). El contador final esperado es `HIJOS × REPETICIONES` exacto, siempre que cada incremento esté protegido por P/V.

▼ ¿Cuál es la diferencia entre la señal que despierta al padre y el semáforo que despierta a los hijos? (control_05)
La señal (`SIGUSR1`) es un mecanismo del sistema operativo que interrumpe directamente a **un solo** proceso —el que tiene registrado el handler— y lo saca de `pause()`. El semáforo, en cambio, es un recurso del kernel que varios procesos pueden consultar de forma independiente: el padre, tras recibir la señal, hace `V()` varias veces (una por hijo bloqueado), y cada hijo —que nunca recibió ninguna señal— simplemente deja de estar bloqueado en su propio `semop()` cuando el valor sube. Son dos mecanismos de "despertar" completamente distintos, encadenados uno después del otro.

▼ ¿Por qué un programa ejecutado con execv() puede volver a localizar la SHM/SEM sin heredar el puntero? (control_06)
Porque `shmget` / `semget` identifican el recurso por una `key` que es visible a nivel de todo el sistema, no por el puntero que devolvió `shmat()` en el proceso original (ese puntero desaparece junto con el código del proceso al hacer `execv`). Cualquier proceso que conozca la misma `key` —aunque no tenga ningún parentesco por `fork()` con quien la creó— puede volver a pedir el mismo recurso y adjuntarse con su propio `shmat()`.

▼ **¿Qué papel cumple cada mecanismo IPC en el control_07 (SHM + SEM + MQ)?**
SHM guarda el **estado** compartido (el contador en sí); el semáforo garantiza la **exclusión mutua** al modificar ese estado; la Message Queue **transporta eventos/resultados** hacia el coordinador (qué proceso vio qué valor, y cuándo), algo que ni la SHM ni el semáforo por sí solos pueden comunicar de forma ordenada y con historial.